

SkillClaw: Collective Skill Evolution for Multi-User LLM Agent Ecosystems

Let skills evolve collectively with an agentic evidential loop

Source: [arXiv:2604.08377](https://arxiv.org/abs/2604.08377)

Agent Skills Remain Static After Deployment

Users repeatedly rediscover the same fixes instead of system-level learning

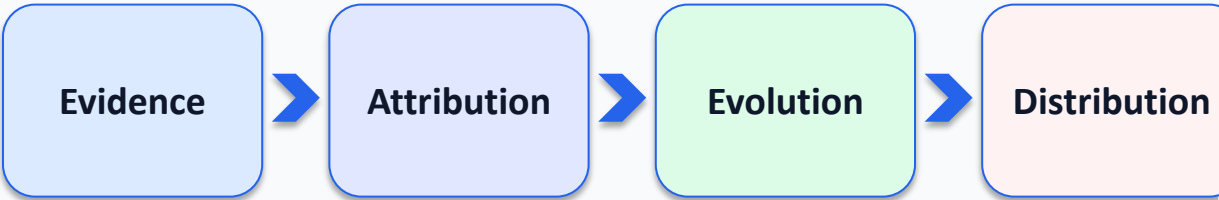
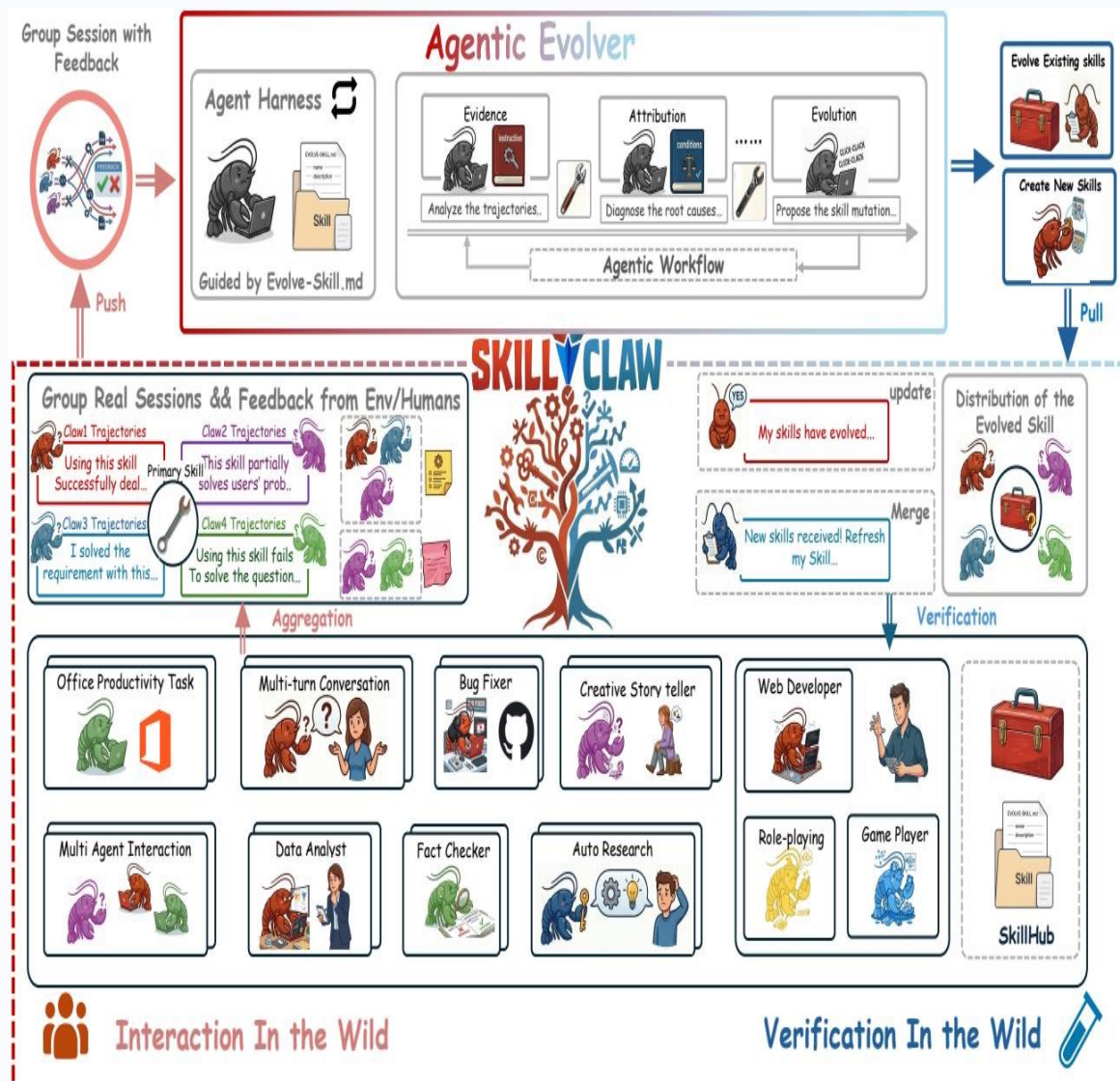
Static paradigm

- Manual skill install from hub
- Runtime trajectories are not converted into updates
- Memory methods stay instance-bound
- Skill libraries do not evolve post-deployment

Observed consequence

- Similar failures recur across users and time
- Recovery strategies remain session-local
- No accumulation of reusable knowledge
- Each user pays repeated exploration cost

Closed-Loop Pipeline: Multi-User Interaction → Shared Skill Evolution



Multi-user sessions provide diverse trajectory evidence
Skill-specific grouping reveals shared failure boundaries
Agentic evolver mutates skills and validator gates updates
Accepted skills synchronize back to all agents

Agentic Evolver: Evidence → Attribution → Evolution

1. Evidence

Analyze grouped trajectories preserving action-feedback causal chains.

2. Attribution

Diagnose root causes behind failures and successes.

3. Evolution

Propose mutations: refine existing skills or create new ones.

Core distinction: updates come from open-ended agent reasoning, not predefined update rules.

Cross-User Trajectory Aggregation Exposes Behavioral Boundaries

Compatible systems: OpenClaw, CoPaw, IronClaw, PicoClaw, ZeroClaw, NanoClaw, NemoClaw

Same skill exercised under diverse user contexts yields complementary success/failure signals

Aggregation distinguishes generalizable improvements from idiosyncratic one-off fixes



Case Study: Multimodal Creative Pipeline — 6-Day Evolution Log

Date/Day	Attempt Outcome	Candidate Skill	Rationale
Day 1	Accepted	validate-tmp-workspace-inputs	Validated /tmp_workspace inputs and environment setup.
Day 2	Rejected	multimodal-input-validation-and-setup	No improvement versus current best pool.
Day 3	Rejected	multimodal-creative-task-pipeline	Candidate extraction pipeline not admitted.
Day 4	Rejected	pipeline (impr.)	Added classification/generation/validation; still not better.
Day 5	Rejected	pipeline (impr.) + validate-required-input-files	Fail-fast file checks added; validator kept current pool.
Day 6	Rejected	pipeline (cand.)	Extended document-to-visual paths; not admitted next cycle.

Result: 1/6 accepted — conservative verification prioritizes regression prevention.

Case Study: Git Push Auth-Fallback — 6-Day Evolution Log

Date/Day	Attempt Outcome	Candidate Skill	Rationale
Day 1	Accepted	git-push-with-auth-fallback	Safe fallback avoids blocking on push failure.
Day 2	Accepted	git-push-with-auth-fallback	Unified filenames reduced inconsistency.
Day 3	Accepted	git-push-with-auth-fallback + git-clone-to-directory	Added auth alternatives + secrets audit fixes.
Day 4	Accepted	(none)	Retest confirmed same best pool remains optimal.
Day 5	Rejected	git-push-with-auth-fallback	Treating push hang as auth failure gave no gain.
Day 6	Rejected	git-push-with-auth-fallback	Identity config/filename consistency gave no gain.

Result: 3/6 accepted updates — iterative refinement succeeded before plateauing.

Key Takeaways: What Distinguishes SkillClaw

1) Collective evolution

Individual interactions improve a shared skill ecosystem over time.

2) Fully automatic

Runtime interaction drives evolution without manual curation.

3) Agentic paradigm

Open-ended reasoning over evidence replaces fixed update rules.

Implication: multi-user agent ecosystems can become continuously improving systems rather than static skill catalogs.